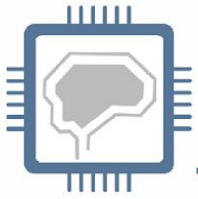


***Inteligência Artificial
e Computacional***

Disciplina: Linguagens de Programação para Inteligência Artificial

Aula 4: Funções

Profa.: Sabrina de A. Silveira



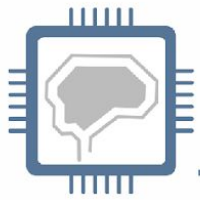
Funções

- As funções são blocos de código que recebem um nome e executam determinada tarefa.
- Quando precisamos realizar a tarefa que foi definida numa função, não é preciso digitar tudo novamente, apenas chamamos a função.
- Para executar a tarefa, as funções podem precisar de dados, que chamamos de parâmetros ou argumentos.
- As funções também podem ou não retornar um ou mais valores ao final da execução.

```
print ( )
```

```
nome = input("Informe seu nome: ")
```

```
print("Olá")
```



Funções

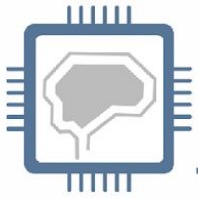
- As funções são blocos de código que recebem um nome e executam determinada tarefa.
- Quando precisamos realizar a tarefa que foi definida numa função, não é preciso digitar tudo novamente, apenas chamamos a função.
- Para executar a tarefa, as funções podem precisar de dados, que chamamos de parâmetros ou argumentos.
- As funções também podem ou não retornar um ou mais valores ao final da execução.

Função print chamada sem parâmetro

➔ `print ( )`

`nome = input("Informe seu nome: ")`

`print("Olá")`



Funções

- As funções são blocos de código que recebem um nome e executam determinada tarefa.
- Quando precisamos realizar a tarefa que foi definida numa função, não é preciso digitar tudo novamente, apenas chamamos a função.
- Para executar a tarefa, as funções podem precisar de dados, que chamamos de parâmetros ou argumentos.
- As funções também podem ou não retornar um ou mais valores ao final da execução.

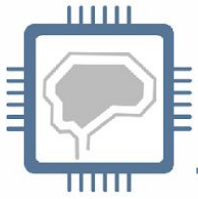
`print ( )`

`nome = input("Informe seu nome: ")`

Função print chamada com um string como parâmetro

→ `print("Olá")`

→ parâmetro



Funções

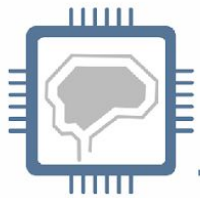
- As funções são blocos de código que recebem um nome e executam determinada tarefa.
- Quando precisamos realizar a tarefa que foi definida numa função, não é preciso digitar tudo novamente, apenas chamamos a função.
- Para executar a tarefa, as funções podem precisar de dados, que chamamos de parâmetros ou argumentos.
- As funções também podem ou não retornar um ou mais valores ao final da execução.

`print ( )`

→ `nome = input("Informe seu nome: ")`

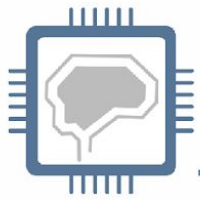
`print("Olá")`

Função input chamada com um string como parâmetro.
Essa função retorna um valor que será armazenado em nome.



Funções

- A linguagem Python nos oferece diversas funções que podemos utilizar em nosso código, como **print** e **input**.
- Podemos também **definir** nossas próprias funções para realizar alguma tarefa. Elas são importantes no reúso de código.
- Uma vez implementada e testada, garantindo o correto funcionamento, uma função pode ser reutilizada ou **chamada** em vários programas, sem que tenhamos que repetir o código.
- Para definir nossas próprias funções usamos a palavra chave **def**.



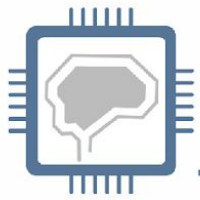
Funções

Apresentamos a definição de uma função simples.

A palavra `def` indica que uma função está sendo definida

```
# definição da função
def greet_user():
    """Imprime saudação na tela"""
    print("Olá!")

# chamada da função
print("Chamando a função greet_user()...")
greet_user()
```



Funções

Apresentamos a definição de uma função simples.

A palavra `def` indica que uma função está sendo definida

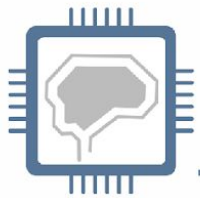
O nome dessa função é `greet_user`

definição da função

```
def greet_user():  
    """Imprime saudação na tela"""  
    print("Olá!")
```

chamada da função

```
print("Chamando a função greet_user()...")  
greet_user()
```



Funções

Apresentamos a definição de uma função simples.

A palavra `def` indica que uma função está sendo definida

O nome dessa função é `greet_user`

Os parênteses vazios indicam que a função não recebe parâmetros

definição da função

```
def greet_user():
```

```
    """Imprime saudação na tela"""
```

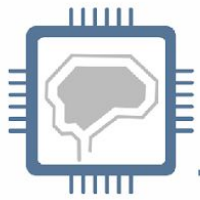
```
    print("Olá!")
```

A linha termina com :

chamada da função

```
print("Chamando a função greet_user()...")
```

```
greet_user()
```



Funções

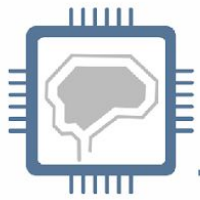
Apresentamos a definição de uma função simples.

As linhas indentadas representam o corpo da função.

O corpo só é executado quando a função é chamada e não quando é definida.

```
# definição da função
def greet_user():
    """Imprime saudação na tela"""
    print("Olá!")

# chamada da função
print("Chamando a função greet_user()...")
greet_user()
```



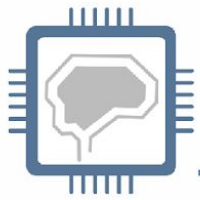
Funções

Apresentamos a definição de uma função simples.

Texto entre 3 aspas é chamado de docstring e descreve o que a função faz. Útil para gerar documentação.

```
# definição da função
def greet_user():
    """Imprime saudação na tela"""
    print("Olá!")

# chamada da função
print("Chamando a função greet_user()...")
greet_user()
```



Funções

Apresentamos a definição de uma função simples.

Texto entre 3 aspas é chamado de docstring e descreve o que a função faz. Útil para gerar documentação.

definição da função

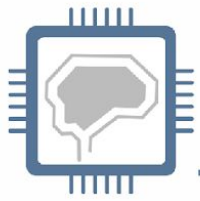
```
def greet_user():  
    """Imprime saudação na tela"""  
    print("Olá!")
```



Único comando executado pela função

chamada da função

```
print("Chamando a função greet_user()...")  
greet_user()
```



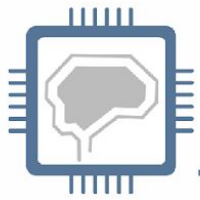
Funções

Apresentamos a definição de uma função simples.

```
# definição da função  
def greet_user():  
    """Imprime saudação na tela"""  
    print("Olá!")
```

Para que o código dentro da função seja executado, é preciso chamar a função.

```
# chamada da função  
print("Chamando a função greet_user()...")  
→ greet_user()
```



Funções

definição da função

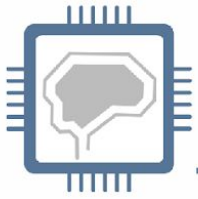
```
def greet_user():  
    """Imprime saudação na tela"""  
    print("Olá!")
```

chamada da função

```
print("Chamando a função greet_user()...")  
greet_user()
```

Saída do programa

Chamando a função greet_user()...
Olá!



Passagem de parâmetro

Veremos um programa que define e usa uma função que requer parâmetros. Esta função é chamada 2 vezes com parâmetros diferentes.

```
# definição da função
```

```
def describe_pet(animal_type, pet_name):
```

```
    """Iprime informação sobre animal de estimação"""
```

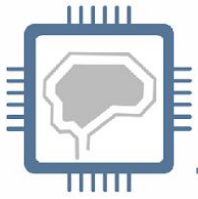
```
    print(f"\nEu tenho um {animal_type}.")
```

```
    print(f"Meu {animal_type} se chama {pet_name.title()}.")
```

```
# função é chamada 2x com parâmetros diferentes
```

```
describe_pet('hamster', 'kiko')
```

```
describe_pet('cachorro', 'billie')
```



Passagem de parâmetro

Definição da função

Nome da função:
describe_pet

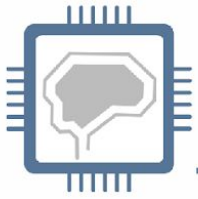
Parâmetros:
animal_type
pet_name

definição da função

→ **def** describe_pet(animal_type, pet_name):
 """Iprime informação sobre animal de estimação"""
 print(f"\nEu tenho um {animal_type}.")
 print(f"Meu {animal_type} se chama {pet_name.title()}.")

função é chamada 2x com parâmetros diferentes

describe_pet('hamster', 'kiko')
describe_pet('cachorro', 'billie')



Passagem de parâmetro

Imprime o string entre
aspas.

Observe que a variável
animal_type será
substituída pelo seu
conteúdo

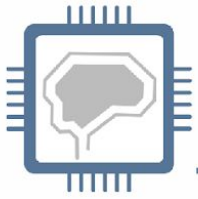


definição da função

```
def describe_pet(animal_type, pet_name):  
    """Imprime informação sobre animal de estimação"""  
    print(f"\nEu tenho um {animal_type}.")  
    print(f"Meu {animal_type} se chama {pet_name.title()}")
```

função é chamada 2x com parâmetros diferentes

```
describe_pet('hamster', 'kiko')  
describe_pet('cachorro', 'billie')
```



Passagem de parâmetro

Imprime o string entre aspas.

Observe que as variáveis **animal_type** e **pet_name** serão substituídas pelo seu conteúdo.

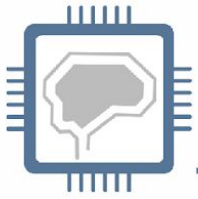
A função title coloca a primeira letra maiúscula.

definição da função

```
def describe_pet(animal_type, pet_name):  
    """Imprime informação sobre animal de estimação"""  
    print(f"\nEu tenho um {animal_type}.")  
    print(f"Meu {animal_type} se chama {pet_name.title()}")
```

função é chamada 2x com parâmetros diferentes

```
describe_pet('hamster', 'kiko')  
describe_pet('cachorro', 'billie')
```



Passagem de parâmetro

definição da função

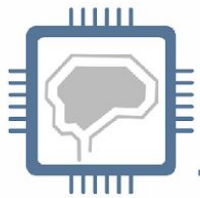
```
def describe_pet(animal_type, pet_name):  
    """Iprime informação sobre animal de estimação"""  
    print(f"\nEu tenho um {animal_type}.")  
    print(f"Meu {animal_type} se chama {pet_name.title()}")
```

Chamadas da função (2)

Observe que os parâmetros devem ser passados conforme a ordem estabelecida na definição da função

função é chamada 2x com parâmetros diferentes

```
{ describe_pet('hamster', 'kiko')  
  describe_pet('cachorro', 'billie')
```



Passagem de parâmetro

definição da função

```
def describe_pet(animal_type, pet_name):  
    """Iprime informação sobre animal de estimação"""  
    print(f"\nEu tenho um {animal_type}.")  
    print(f"Meu {animal_type} se chama {pet_name.title()}.")
```

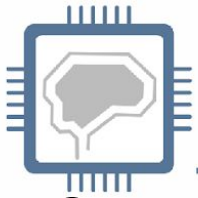
função é chamada 2x com parâmetros diferentes

```
describe_pet('hamster', 'kiko')  
describe_pet('cachorro', 'billie')
```

Saída do programa

Eu tenho um hamster.
Meu hamster se chama Kiko.

Eu tenho um cachorro.
Meu cachorro se chama Billie.



Retorno de valores

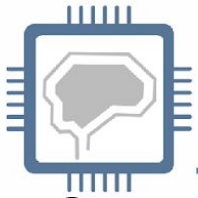
Os exemplos vistos executam alguma tarefa (imprimem na tela) mas não retornam valor. Veremos agora uma função que retorna valor. Dado um string (mytext), essa função repete o string por um determinado número de vezes (text_length)

Definição da função.
Aqui a função **não** é executada.

```
def repeatStr(mytext, text_length):  
    s = ''  
    for i in range(0, text_length):  
        s = s + mytext  
    return s
```

Chamada da função.
Aqui a função é executada.

```
# chamada da função  
output = repeatStr('->', 20)  
  
print(output)
```



Retorno de valores

Os exemplos vistos executam alguma tarefa (imprimem na tela) mas não retornam valor. Veremos agora uma função que retorna valor. Dado um string (mytext), essa função repete o string por um determinado número de vezes (text_length)

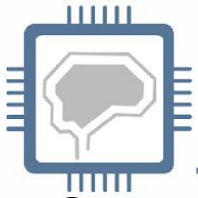
Definição da função

Nome da função:
repeatStr

Parâmetros:
mytext
text_length

```
→ def repeatStr(mytext, text_length):  
    s = ''  
    for i in range(0, text_length):  
        s = s + mytext  
    return s
```

```
# chamada da função  
output = repeatStr('->', 20)  
  
print(output)
```



Retorno de valores

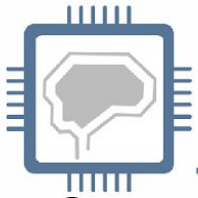
Os exemplos vistos executam alguma tarefa (imprimem na tela) mas não retornam valor. Veremos agora uma função que retorna valor. Dado um string (mytext), essa função repete o string por um determinado número de vezes (text_length)

Inicializa **s** com string vazio



```
def repeatStr(mytext, text_length):  
    s = ''  
    for i in range(0, text_length):  
        s = s + mytext  
    return s
```

```
# chamada da função  
output = repeatStr('->', 20)  
  
print(output)
```



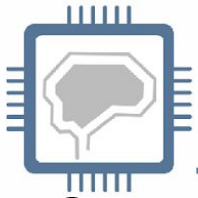
Retorno de valores

Os exemplos vistos executam alguma tarefa (imprimem na tela) mas não retornam valor. Veremos agora uma função que retorna valor. Dado um string (mytext), essa função repete o string por um determinado número de vezes (text_length)

O for vai de 0 até **text_length-1**, de modo que o comando dentro do for é repetido **text_length** vezes.

```
def repeatStr(mytext, text_length):  
    s = ''  
    for i in range(0, text_length):  
        s = s + mytext  
    return s
```

```
# chamada da função  
output = repeatStr('->', 20)  
  
print(output)
```



Retorno de valores

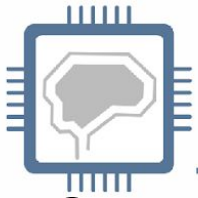
Os exemplos vistos executam alguma tarefa (imprimem na tela) mas não retornam valor. Veremos agora uma função que retorna valor. Dado um string (mytext), essa função repete o string por um determinado número de vezes (text_length)

O string em **s** será concatenado com o string em **mytext** e o resultado será armazenado novamente em **s**.



```
def repeatStr(mytext, text_length):  
    s = ''  
    for i in range(0, text_length):  
        s = s + mytext  
    return s
```

```
# chamada da função  
output = repeatStr('->', 20)  
  
print(output)
```



Retorno de valores

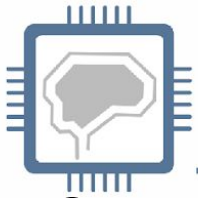
Os exemplos vistos executam alguma tarefa (imprimem na tela) mas não retornam valor. Veremos agora uma função que retorna valor. Dado um string (mytext), essa função repete o string por um determinado número de vezes (text_length)

Após text_length iterações, a função retorna o string s.



```
def repeatStr(mytext, text_length):  
    s = ''  
    for i in range(0, text_length):  
        s = s + mytext  
    return s
```

```
# chamada da função  
output = repeatStr('->', 20)  
  
print(output)
```



Retorno de valores

Os exemplos vistos executam alguma tarefa (imprimem na tela) mas não retornam valor. Veremos agora uma função que retorna valor. Dado um string (mytext), essa função repete o string por um determinado número de vezes (text_length)

Aqui ocorre a chamada à função **repeatStr**.

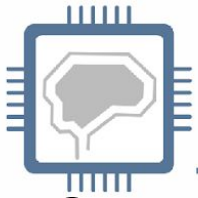
A função é chamada com o string **'->'** e com o inteiro **20** como parâmetros.

O retorno da função, que é o string **'->'** repetido **20** vezes é armazenado na variável **output**.

```
def repeatStr(mytext, text_length):  
    s = ''  
    for i in range(0, text_length):  
        s = s + mytext  
    return s
```

chamada da função

```
→ output = repeatStr('->', 20)  
  
print(output)
```



Retorno de valores

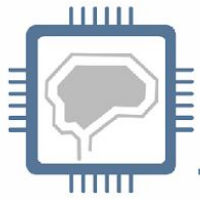
Os exemplos vistos executam alguma tarefa (imprimem na tela) mas não retornam valor. Veremos agora uma função que retorna valor. Dado um string (mytext), essa função repete o string por um determinado número de vezes (text_length)

```
def repeatStr(mytext, text_length):  
    s = ''  
    for i in range(0, text_length):  
        s = s + mytext  
    return s
```

```
# chamada da função  
output = repeatStr('->', 20)
```

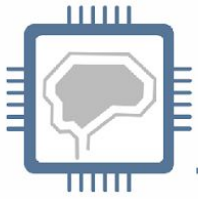
Imprime a variável output na tela

→ `print(output)`



Definições locais e globais

- Variáveis criadas fora de funções são chamadas **variáveis globais**. Elas podem ser usadas dentro e fora de funções.
- Se criamos dentro de uma função uma variável com o mesmo nome de outra variável que já existe fora função, essa nova variável terá **escopo local** ao programa. A variável global de mesmo nome continuará global e com o valor original.



Definições locais e globais 1

Definição da função myfunc()

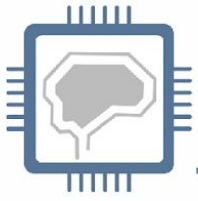


definição da função

```
def myfunc():  
    x = "fantástico"  
    print("Escopo local...")  
    print("Python é " + x)
```

programa principal

```
x = "impressionante"  
myfunc()  
print("\nEscopo global...")  
print("Python é " + x)
```



Definições locais e globais 1

Definição da função myfunc()

Definição da variável local x

definição da função

def myfunc():

→ **x** = "fantástico"
print("Escopo local...")
print("Python é " + x)

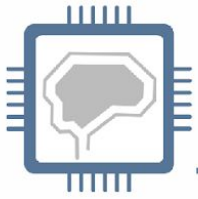
programa principal

x = "impressionante"

myfunc()

print("\nEscopo global...")

print("Python é " + x)



Definições locais e globais 1

Definição da função myfunc()

Definição da variável local x

Imprime string na tela

definição da função

def myfunc():

 x = "fantástico"

→ print("Escopo local...")

print("Python é " + x)

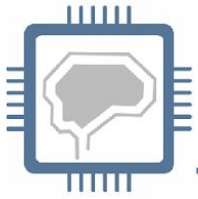
programa principal

x = "impressionante"

myfunc()

print("\nEscopo global...")

print("Python é " + x)



Definições locais e globais 1

Definição da função myfunc()

Definição da variável da variável local x

Imprime string na tela

Imprime o conteúdo da variável local na tela

definição da função

def myfunc():

 x = "fantástico"

 print("Escopo local...")

→ print("Python é " + x)

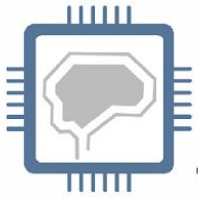
programa principal

x = "impressionante"

myfunc()

print("\nEscopo global...")

print("Python é " + x)



Definições locais e globais 1

definição da função

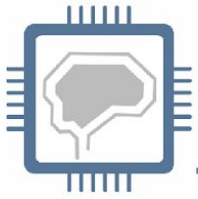
```
def myfunc():  
    x = "fantástico"  
    print("Escopo local...")  
    print("Python é " + x)
```

Definição da variável global x



programa principal

```
x = "impressionante"  
myfunc()  
print("\nEscopo global...")  
print("Python é " + x)
```



Definições locais e globais 1

definição da função

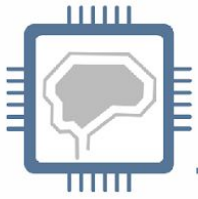
```
def myfunc():  
    x = "fantástico"  
    print("Escopo local...")  
    print("Python é " + x)
```

Definição da variável global x
Chama a função myfunc



programa principal

```
x = "impressionante"  
myfunc()  
print("\nEscopo global...")  
print("Python é " + x)
```



Definições locais e globais 1

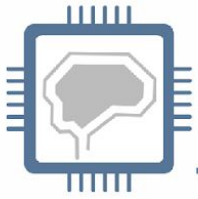
definição da função

```
def myfunc():  
    x = "fantástico"  
    print("Escopo local...")  
    print("Python é " + x)
```

Definição da variável global x
Chama a função myfunc
Imprime string na tela

programa principal

```
x = "impressionante"  
myfunc()  
→ print("\nEscopo global...")  
print("Python é " + x)
```



Definições locais e globais 1

definição da função

```
def myfunc():  
    x = "fantástico"  
    print("Escopo local...")  
    print("Python é " + x)
```

Definição da variável global x

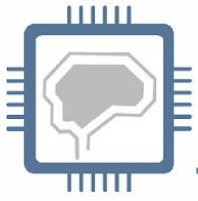
Chama a função myfunc

Imprime string na tela

Imprime o conteúdo da variável global na tela

programa principal

```
x = "impressionante"  
myfunc()  
print("\nEscopo global...")  
→ print("Python é " + x)
```



Definições locais e globais 1

definição da função

```
def myfunc():  
    x = "fantástico"  
    print("Escopo local...")  
    print("Python é " + x)
```

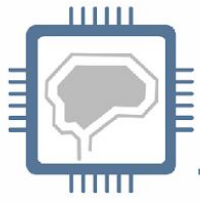
programa principal

```
x = "impressionante"  
myfunc()  
print("\nEscopo global...")  
print("Python é " + x)
```

Saída do programa

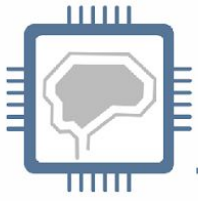
Escopo local...
Python é fantástico

Escopo global...
Python é impressionante



Definições locais e globais 2

- Quando criamos uma variável dentro de uma função ela é, em geral, local e portanto usada dentro da função.
- É possível **criar uma variável global dentro de uma função** usando a palavra chave **global**.
- O uso de variáveis globais é **desencorajado**.



Definições locais e globais 2

Dentro da função myfunc criamos uma variável global chamada x

definição da função

```
def myfunc():
```

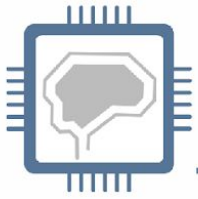
→ **global** x

```
    x = "fantástico"
```

programa principal

```
myfunc()
```

```
print("Python é " + x)
```



Definições locais e globais 2

Dentro da função myfunc criamos
uma variável global chamada x

Um valor é atribuído a x dentro da função

definição da função

```
def myfunc():
```

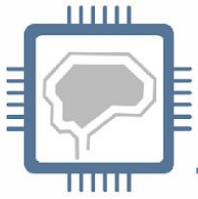
```
    global x
```

```
    → x = "fantástico"
```

programa principal

```
myfunc()
```

```
print("Python é " + x)
```



Definições locais e globais 2

Dentro da função myfunc criamos
uma variável global chamada x

Um valor é atribuído a x dentro da função

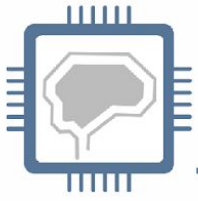
definição da função

```
def myfunc():  
    global x  
    x = "fantástico"
```

A função myfunc é chamada e cria a variável global x →

programa principal

```
myfunc()  
print("Python é " + x)
```



Definições locais e globais 2

Dentro da função myfunc criamos uma variável global chamada x

Um valor é atribuído a x dentro da função

A função myfunc é chamada e cria a variável global x

Imprime o valor da variável global na tela

definição da função

```
def myfunc():
```

```
    global x
```

```
    x = "fantástico"
```

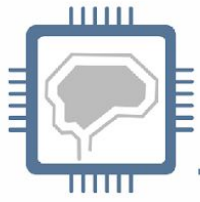
programa principal

```
myfunc()
```

```
print("Python é " + x)
```

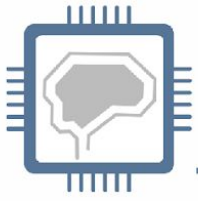
Saída do programa

Python é fantástico



Definições locais e globais 3

- É possível **mudar o valor de uma variável global dentro de uma função** utilizando a palavra chave global.
- É uma boa prática de programação evitar o uso de variáveis globais dentro de funções devido ao risco de efeitos colaterais e perda da generalidade.

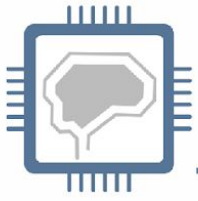


Definições locais e globais 3

A variável global x recebe o valor fantástico

```
# definição da função  
def myfunc():  
    { global x  
      x = "fantástico"
```

```
# programa principal  
x = "incrível"  
myfunc()  
print("Python é " + x)
```



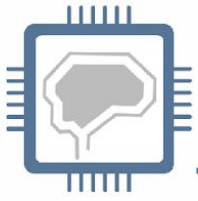
Definições locais e globais 3

A variável global x recebe o valor "fantástico"

```
# definição da função  
def myfunc():  
    global x  
    x = "fantástico"
```

A variável global x recebe o valor "incrível" e a função myfunc é chamada

```
# programa principal  
{ x = "incrível"  
  myfunc()  
  print("Python é " + x)}
```



Definições locais e globais 3

A variável global x recebe o valor "fantástico"

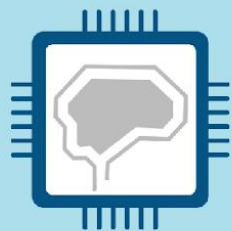
```
# definição da função  
def myfunc():  
    global x  
    x = "fantástico"
```

A variável global x recebe o valor "incrível" e a função myfunc é chamada

```
# programa principal  
x = "incrível"  
myfunc()  
print("Python é " + x)
```

Saída do programa

Python é fantástico



Inteligência Artificial e Computacional

Profa.: Sabrina de A. Silveira